LESSON PLAN · PYTHON (GR 6–8) BEGINNER: PYTHON GAME CODERS

Lesson 1: Hello, Python!

Module 1 | 60 minutes | print, strings, comments | Grades 6–10 · ages 11–15

Objective: Write your first Python programs with print(), strings and comments, and make text art that shows on screen.

Success criteria

- I can use print() to show text on the screen.
- I can explain what a string is and close every quote.
- I can write a comment with #.
- I can fix a simple syntax error by reading the message.

Key vocabulary

- **program**: A list of instructions the computer follows in order.
- **print()**: The command that shows text on the screen.
- **string**: Text inside quote marks, like "Hello".
- **comment**: A note that starts with #. Python skips it.
- **syntax error**: A typing mistake that stops Python from understanding the code.
- **output**: What the program shows on the screen when it runs.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: name_badge.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Robot Printer**	Play Robot Printer (unplugged). Learners act out print and comments. Celebrate the best robot.
5 min	**Hook and video**	Show the hook: print("Hello, Kitabash!"). Thumbs poll: “Who has typed real code before?” Play the video with captions and pause at “Spot the bug”.
10 min	**Live demo and build-along**	Run the live demo exactly as written, then everyone types the name badge build-along with you. Thumbs check after each step.
8 min	**Challenge and predict**	Learners do “three-line hello”, then the predict-the-output and match activities. Quick competition: first three correct predictions shout “Kabaddi!”.

15 min	**Project in pairs**	Pair programming in breakout rooms: one types (driver), one checks quotes and brackets (navigator); swap every 5 minutes. Build the badge or scoreboard poster.
6 min	**Debug race**	Race to fix “the broken welcome screen”. The first learner to explain all three bugs wins a shout-out.
7 min	**Show-and-tell**	Three volunteers share their screen and run their poster. Class gives two stars and a wish in the chat.
4 min	**Wrap-up and exit ticket**	Recap the four success criteria with a fist-to-five poll. Exit ticket: print “I am a Python coder!”.

Unplugged warm-up

Robot Printer (5 min game): one learner is the “Python robot”. Others hold up cards with instructions such as print("Namaste") or # print("Dance"). The robot must say out loud ONLY what a real print would show, and stay silent for comments. Anyone who catches the robot making a mistake becomes the next robot.

Teaching the idea

Python reads your program from top to bottom and follows each line exactly. print() shows whatever is inside its brackets. Text must go inside matching quotes, and that text is called a string. Lines starting with # are comments: Python ignores them, so we use them for notes. Triple quotes let one string cover many lines, which is how we draw text art.

Analogy: print() is like a loudspeaker: whatever you put in front of it gets announced exactly as written, spelling mistakes and all.

Live demo: 1) Open the Python Lab and share your screen. 2) Type print("Hello, Kitabash!") and press Run; point to the output panel. 3) Delete the closing quote, press Run, and read the red error aloud: “unterminated string literal” means the quote never ended. Put the quote back. 4) Type Print with a capital P, run it, and show the NameError; fix it. 5) Add # My first program on line 1 and run: point out that nothing new appears. 6) Type print(""" , press Enter, type (o o) and | | on two lines, then """) and run to show the art keeps its spaces.

Questions to ask

Ask	Listen for
What does print() do?	It shows the text inside the brackets on the screen.
Why does print("Hi) give an error?	The closing quote is missing, so Python does not know where the string ends.
What happens to a line starting with #?	Python skips it; it is a comment for humans.
How do you print three lines?	Use three print commands, or one triple-quoted string.

Is Print("Hi") the same as print("Hi")?	No. Python is case-sensitive, so Print is an error.

Differentiation

Support
Give a printed “bracket and quote buddy” card: every (needs) and every " needs ". Let learners start from the starter file and only change the words.

Stretch
Make a 10-line poster with two different borders, and use print("=" * 30) to draw a border of 30 equals signs in one line.

Common misconceptions

Misconception	How to fix it
The computer will understand print("Hi) because it is close enough	Python needs exact syntax. One missing quote stops the whole program.
Comments are shown on the screen	Comments are skipped. Only print shows things on the screen.
Print and print are the same	Python is case-sensitive: always use lower-case print.
The quotes are printed too	Quotes only mark where the string starts and ends; they are not shown.

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
Write one line of Python that prints: I am a Python coder!

Homework
Make a birthday card program for someone at home: a border, their name, and a triple-quote cake picture. Run it for them!

Answer key — Kitabash activities

1. Python Lab: three-line hello (perform · python lab)

Goals checked: Prints the three lines in order · Has a comment · Runs with no errors

One solution:

```
# My first program
print("Hello, Python!")
print("My name is Kabir.")
print("I like biryani and badminton.")
```

2. Predict the output: comments (perform · code output)

3. Match each piece of Python to its job (perform · matching)

print("Hi") → Shows Hi on the screen · "Hi" → A string: text in quotes · # Hi → A comment that Python skips · """Hi""" → A string that can cover many lines

4. Project: name badge or scoreboard poster (perform · python lab)

Goals checked: Prints at least 6 lines · Uses print and comments · Runs with no errors

Example program:

```
# Cricket scoreboard poster
# Made by: Meera

print("==============================")
print("   KITABASH CRICKET CUP 2026  ")
print("==============================")
print("   Mumbai Waves   vs   Delhi Kites")
print("------------------------------")
print("   Mumbai Waves : 182 / 4")
print("   Delhi Kites  : 179 / 8")
print("------------------------------")
print("""
     WINNER: MUMBAI WAVES!

         ___________
        '._==_==_=_.'
        .-:      :-.
        '-|:     |-'
          ':.   .:'
            ) (
          .'   '.
         '-------'
""")
print("Player of the match: Riya (5 wickets)")
```

5. Debug it: the broken welcome screen (probe · python lab)

Goals checked: Runs with no errors · Prints all three lines

One solution:

```
print("Welcome to the match!")
print("India vs Australia")
print("Toss: India won")
```

Answer key — worksheet

1. **Key words:** program = A list of instructions the computer follows in order. · print() = The command that shows text on the screen. · string = Text inside quote marks, like "Hello". · comment = A note that starts with #. Python skips it.
2. **Plan your program:** Goals: Prints the three lines in order; Has a comment; Runs with no errors. One solution: # My first program print("Hello, Python!") print("My name is Kabir.") print("I like biryani and badminton.")
3. **Predict the output: comments:** A. Cricket and Kabaddi on two lines
4. **Match each piece of Python to its job:** print("Hi") → Shows Hi on the screen · "Hi" → A string: text in quotes · # Hi → A comment that Python skips · """Hi""" → A string that can cover many lines
5. **Plan your program:** Goals: Runs with no errors; Prints all three lines. One solution: print("Welcome to the match!") print("India vs Australia") print("Toss: India won")

Kitabash original curriculum.